

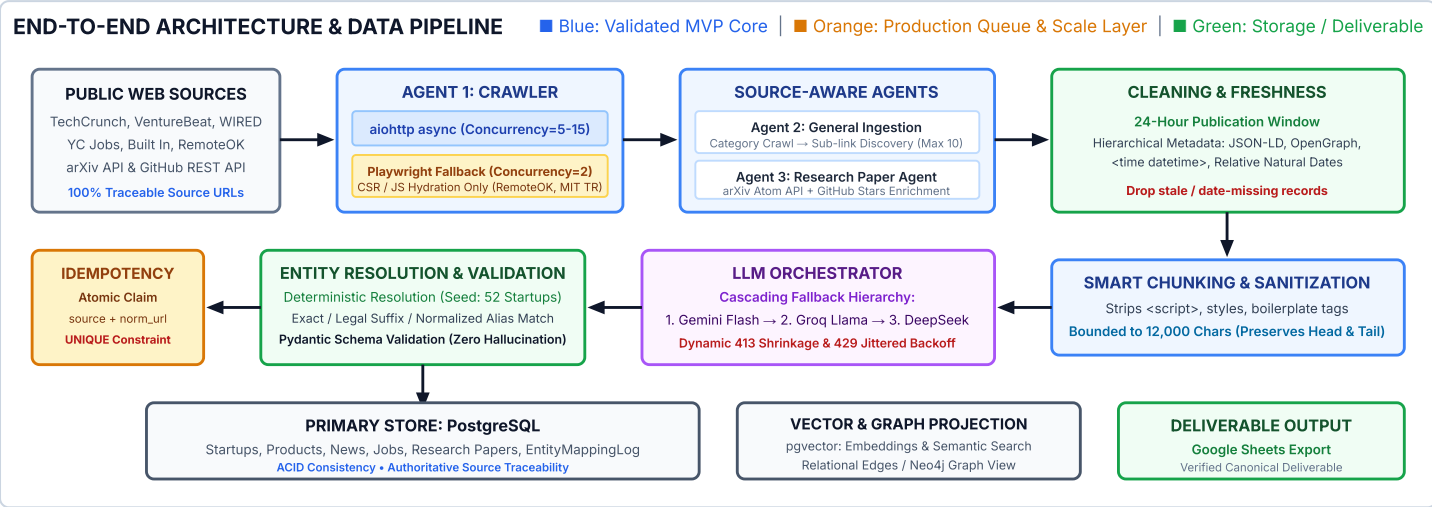
AI Intelligence Pipeline: Architecture & 500K Scale Design

PRODUCTION SPECIFICATION & LOCAL VERIFICATION REFERENCE

VALIDATED CORE MVP

PRODUCTION TOPOLOGY

The **AI Intelligence Pipeline** is a high-throughput, fault-tolerant ingestion and synthesis architecture engineered to extract, canonicalize, and validate structured intelligence on startups, products, jobs, and research papers from heterogeneous web sources. It theoretically scales to **500,000+ records** without altering core application logic, combining bounded asynchronous concurrency, persistent distributed-safe idempotency, and cascading LLM fallback resilience.



CURRENT IMPLEMENTED MVP VS. PRODUCTION SCALE ARCHITECTURE

Architectural Subsystem	Current Implemented MVP (Validated in Codebase)	Production-Scale Design (500,000+ Target)
Crawler & Ingestion	<code>aiohttp</code> async pool (HTTP limit=5-15) + Playwright fallback (limit=2) for JS-rendered pages. Bounded batch execution via <code>fetch_many_batched()</code> .	Distributed worker fleet (e.g., Celery/Ray/Kubernetes pods) consuming from URL partition queues; worker autoscaling based on queue depth.
Work Scheduling	Asyncio coroutine tasks partitioned into bounded in-memory batches of 100 items (O(batch_size) memory ceiling).	Durable distributed queue (Kafka / RabbitMQ / Redis Streams) partitioned by source domain with adaptive per-domain token bucket rate limiting.
Idempotency Store	<code>src/storage/idempotency.py</code> : SQLite WAL mode with atomic <code>UNIQUE(source, normalized_url)</code> constraints and thread-safe connections.	Shared PostgreSQL (<code>INSERT ... ON CONFLICT DO NOTHING</code>) or Redis cluster (<code>SET ... NX EX 86400</code>) accessible across all distributed worker nodes.
LLM Extraction	Cascading provider fallback: Gemini Flash → Groq Llama → DeepSeek. Bounded 12,000 char input, 413 dynamic halving, 429 backoff with jitter.	Centralized LLM gateway with Redis token bucket rate tracking, asynchronous completion worker pools, and fallback provider routing.
Entity Resolution	<code>DeterministicEntityResolver</code> : seed dictionary (52 AI startups/products), legal suffix stripping, alias tables, explainable mapping logs.	Distributed entity matching pipeline caching canonical dictionaries in memory with automated alias learning and human-in-the-loop review queues.
Persistence Layer	SQLite local persistence + direct Google Sheets API / CSV export. Raw source HTML preservation in <code>CrawlResult</code> .	PostgreSQL cluster (read-replicas + write primary) for canonical records; pgvector for semantic search; relational edges for graph querying.

Core Engineering Guarantee: Non-Negotiable Source Traceability

Every record generated retains immutable origin metadata: `source_url` (category/board listing), `item_url` (exact normalized article/job permalink), and `collected_at` (UTC ISO timestamp). **LLMs are strictly restricted to structured extraction** and are never permitted to fabricate URLs, invent dates, or hallucinate canonical entity existence. If metadata is missing or unverified, fields remain strictly `null`.

500K Scale Strategy, Rate Handling & Fault Tolerance

CONCURRENCY BOUNDS • PAYLOAD OPTIMIZATION • DISTRIBUTED RESILIENCE

BENCHMARK: 718.4 TASKS/SEC

1. 500,000+ RECORD SCALABILITY & BOUNDED CONCURRENCY

Scaling to 500,000+ items without application code modification requires decoupling workload magnitude from memory and task concurrency. Naive implementations using `asyncio.gather(*500000_tasks)` immediately cause catastrophic failure: event loop starvation, hundreds of megabytes of leaked coroutine stack frames, and socket exhaustion.

Implemented Bounded Batching Pattern: As verified in `src/crawler/base.py`, the system processes workloads in discrete batches of `batch_size=100` guarded by an internal `asyncio.Semaphore(concurrency=5..15)`:

```
WORKLOAD (500k URLs)
└─ Bounded Batches (N=100) [0(batch_size) Memory Ceiling]
   └─ Concurrency Semaphore (K=10) [Guaranteed Active Sockets ≤ K]
      └─ Atomic Idempotency Claim → aiohttp fetch → Next Batch
```

Horizontal Scaling Model: In a production cluster, 500,000 target URLs are partitioned across \$W\$ independent crawler worker containers consuming from a shared queue (e.g., Redis Streams / RabbitMQ). Each worker node runs the existing bounded crawler:

- Zero Inter-Worker Locking:** Coordination is achieved purely via atomic database claims (`claim_if_unseen`).
- Linear Scale-Out:** Aggregate crawl capacity scales linearly (\$W \times \text{Worker Capacity}\$) without modifying any extraction or agent logic.
- Isolated Failure Domains:** A crash in Worker \$i\$ does not affect Worker \$j\$. Unfinished claims expire via TTL and are safely requeued.

Step 23 Empirical Load Validation (Local Synthetic Transport Benchmark)					100% Task Accounting • Zero Concurrency Leaks
Workload Size	Elapsed Time	Measured Throughput	Configured Concurrency	Max Observed Concurrency	Concurrency Bounding
100 Tasks	0.272 s	367.5 tasks/sec	5	5	PASS (Strictly Respected)
1,000 Tasks	1.392 s	718.4 tasks/sec	10	10	PASS (Strictly Respected)
5,000 Tasks	2.934 s	1,704.4 tasks/sec	15	15	PASS (Strictly Respected)

Theoretical Extrapolation: At 718.4 tasks/sec, 500,000 crawl operations require ~696 seconds (~11.6 minutes). *Caveat:* This measures internal event loop capacity and pipeline bounding. Real-world crawls are governed by domain rate limits, network RTT, and LLM throughput.

2. HTTP VS. PLAYWRIGHT HEADLESS BROWSER WORKLOAD SEGREGATION

Asynchronous HTTP (aiohttp) — Primary Path (>95% Volume)	Headless Browser (Playwright) — Fallback Path (<5% Volume)
- Throughput: 500 – 2,000 tasks/sec per modern server core. - Resource footprint: ~2 KB memory per connection; negligible CPU load. - Utilized for: RSS, APIs, clean SSR HTML (TechCrunch, YC, Wired, arXiv).	- Throughput: 0.5 – 2.0 tasks/sec per browser tab instance. - Resource footprint: ~150 MB RAM per Chromium context; high CPU overhead. - Strictly restricted to: Known CSR boards (RemoteOK, WorkingNomads, MIT TR).

3. EXACT HTTP 413 AND 429 HANDLING STRATEGIES

HTTP 413: Payload Too Large Strategy

Root Cause: Raw web pages exceed provider token/context window limits. Handled fundamentally differently from 429 because retrying identical payloads will permanently fail.

- Proactive Sanitization:** Strip `<script>`, `<style>`, SVGs, and inline CSS; collapse whitespace. Clean text is pre-truncated to `12,000 characters`.
- Head + Tail Context Anchoring:** Truncation retains the first 7,000 chars (headlines, leads) and final 4,500 chars (author, dates, disclosures).
- Reactive 413 Interception:** If provider responds with HTTP 413, intercept error, dynamically halve payload to 6,000 chars, and re-attempt.
- Provider Cascading:** If 413 persists, cascade to the next fallback provider. Source URLs and metadata remain intact.

HTTP 429: Rate & Quota Throttling Strategy

Root Cause: Temporary rate limit saturation on source websites (crawler) or upstream LLM provider quotas. Handled via temporal backoff.

- Crawler 429 Protocol:** Inspect `Retry-After` header. If present, sleep for the requested duration (capped at 60s); otherwise record 429 cleanly without hammering the site.
- LLM 429 Protocol:** If Gemini Flash throttles (HTTP 429), inspect `Retry-After` or apply exponential backoff + jitter: $1.5^{\text{attempt}} \times \text{jitter}$.
- Bounded Retries & Cascading:** Max 3 retries. If exhausted, instantly switch to next provider: **Gemini Flash → Groq Llama → DeepSeek**.
- Failure Isolation:** Exhausted 429s record structured errors. Never block or cancel unrelated pipeline records.

4. COMPREHENSIVE FAULT ISOLATION MATRIX

Condition	Crawler Action	LLM Orchestrator Action	System Isolation Guarantee
HTTP 403 Forbidden	Record failure immediately; 0 retries (polite crawling; no bypass).	N/A (Crawl phase terminated).	Single task records error; remaining 999 tasks execute uninterrupted.
HTTP 429 Rate Limit	Honor <code>Retry-After</code> (up to 60s); record failure if unrecoverable.	Exponential backoff with jitter; cascade to Groq Llama → DeepSeek.	Prevents IP bans; provider failover transparent to downstream agents.
HTTP 5xx Server Error	Retry up to 3 times with exponential backoff ($\text{base}=1.5s$).	Retry up to 3 times; cascade to next provider.	Transient upstream drops recover automatically; permanent failures isolated.
Timeout & Connection Drop	Catch <code>TimeoutError</code> (20s limit); retry bounded; mark failed.	Catch socket timeout (30s limit); switch provider.	No socket leaks or hung processes; event loop remains responsive.

1. FRESHNESS VS. DISTRIBUTED IDEMPOTENCY (DECOUPLED CONCEPTS)

Freshness Evaluation ("Is it chronologically recent?")

Evaluated strictly against published timestamps via `src/agents/freshness.py`. Only items with verified dates within the trailing 24 hours (`[t_{\text{now}} - 24\text{h}, t_{\text{now}}]`) are admitted. Missing or unverified publication dates are dropped to prevent stale ingestion.

Distributed Idempotency ("Has it been claimed/processed?")

Evaluated on stable identity: `source + normalize_storage_url(item_url)`. Strips marketing tracking parameters (`utm_*`, `ref`) and fragments. Prevents double-crawling across distributed nodes even if identical articles are scheduled concurrently.

Atomic Claim Race Condition Resolution: When Worker A and Worker B encounter the same URL simultaneously, both execute an atomic claim against the shared store. In the validated MVP (SQLite), this is enforced by `UNIQUE(source, normalized_url)`. In production PostgreSQL, it executes `INSERT INTO idempotency_claims (...) VALUES (...) ON CONFLICT DO NOTHING RETURNING id;`. Exactly **ONE worker wins the claim**; the losing worker receives `False` and safely discards the item. *Content modifications on an existing URL do not create duplicate records.*

2. STORAGE ARCHITECTURE: PRIMARY DB, VECTOR & GRAPH PROJECTIONS

A. Primary Database (PostgreSQL)

System of Record: Relational PostgreSQL with full ACID guarantees. Normalized schema tables:

- `startups`: canonical name, domain, funding, employee count, status.
- `products`: canonical product, startup_id FK, categories.
- `jobs`: role title, company_id FK, location, remote flag, date.
- `news`: title, source_url, item_url, published_date, summary.
- `research_papers`: arXiv ID, title, authors, GitHub repo, stars.
- `entity_mapping_logs`: raw name, canonical name, match method, confidence score.

B. Vector Storage (pgvector)

Semantic Retrieval Layer: Implemented within PostgreSQL using the `pgvector` extension.

- Embeddings generated via `text-embedding-3-small` or open-source BGE embeddings.
- Tables: `paper_embeddings` (1536-dim abstract vectors), `startup_embeddings`.
- HNSW / IVFFlat indexing for fast nearest-neighbor similarity search (<10ms).
- Enables clustering of competing startups and semantic paper recommendations.

C. Graph Projection (Intelligence Graph)

Multi-Hop Relationship Layer:

- Option A (Relational Edges):** Foreign key junction tables (`startup_products`, `paper_startups`, `founder_startups`) queried via recursive CTEs.
- Option B (Scale-Out Graph DB):** Event-driven change data capture (CDC) streaming edges to **Neo4j** for rich graph querying:
`(Startup)-[:DEVELOPS]→(Product)`
`(Startup)-[:RESEARCHED]→(Paper)`
`(News)-[:MENTIONS]→(Startup)`

3. RESEARCH PAPER ENRICHMENT FLOW & BEST-EFFORT GITHUB LINKING

Research papers follow a deterministic, non-LLM acquisition pipeline verified in `src/agents/agent3_research.py`:

arXiv Atom API (Search AI/CS queries) → XML Stream Parsing (title, authors, published, abstract, arXiv URL)
↳ Hugging Face Papers API / Papers with Code API (Exact paper-to-code mapping resolution)
↳ GitHub REST API (Fetch real-time repository star count, open issues, primary language)
↳ Output Canonical Paper Record [If GitHub unverified → `github_url=null`; ZERO hallucination]

4. THEORETICAL CAPACITY VS. REAL-WORLD PRODUCTION BOTTLENECKS

Capacity Mathematical Projection: While local synthetic benchmarks demonstrate engine capacity of **718.4 tasks/sec** (theoretically crawling 500k records in 11.6 minutes), production deployments encounter severe real-world physical constraints:

Subsystem	Synthetic Test	Real-World Production Reality
Source Ingestion	0.3 ms local delay	100–300 ms network RTT; 1–5 req/sec polite limit per domain.
Browser Render	Bypassed (mock)	2,000–5,000 ms per page load + DOM hydration (Playwright).
LLM Extraction	Pre-cached / mock	500–1,500 ms generation RTT; strict provider TPM/RPM quotas.
Database Writes	In-memory SQLite	Disk I/O, transaction commit latency, index write overhead.

Production Sizing for 500,000 Records:

- Assuming polite crawling across 500 distinct host domains with 2 concurrent connections each (\$1,000\$ active requests) yielding an aggregate web crawl rate of **~100 pages/sec**:
$$\text{Crawl Time} = \frac{500,000}{100} = 5,000 \text{ s} \approx 1.38 \text{ hours}$$
- Assuming 10% of crawled pages qualify under 24-hour freshness (\$50,000\$ LLM extractions) distributed across 3 API providers at 30 req/sec aggregate:
$$\text{Extraction Time} = \frac{50,000}{30} = 1,666 \text{ s} \approx 27.8 \text{ minutes}$$
- Conclusion:** With horizontal crawler worker fleet, the complete 500,000-record pipeline completes in **under 2 hours** without any modifications to application code.

5. SECURITY, ETHICS & ACCESS CONTROL ARCHITECTURE

Strict Access Compliance: The pipeline strictly honors web standards and ethical scraping policies: (1) Respects `robots.txt` and source rate limits; (2) Zero CAPTCHA or Cloudflare bypass mechanisms; (3) Identifies with clean User-Agent headers; (4) API keys (Gemini, Groq, DeepSeek) are managed exclusively through environment variables and never logged or committed; (5) Google Sheets output acts solely as the user deliverable layer, keeping internal storage decoupled and secure.